

OOP Practice Problem

1. Student Information Using Class and Object

A university wants to store basic information about its students. For this purpose, create a Java class named Student that contains two data members: name and id. Create an object of the Student class in the main() method and assign suitable values to these data members. Finally, display the student's ID and name.

Task: Write a Java program that demonstrates how a class is created and how an object is used to access the data members of the class.

2. Book Information Using Multiple Objects

A library needs a simple program to store information about its books. Create a Java class named Book containing two data members: title and price. In the main() method, create two different objects of the Book class and assign different titles and prices to them.

Display the information of both books separately.

Task: Write a Java program to demonstrate how multiple objects of the same class can contain different data.

3. Addition Using a Method

A student wants to create a simple Java program that performs addition using a separate method. Create a class named Calculator containing a method called add().

The method should:

- Receive two integer numbers as parameters.
- Calculate their sum.
- Return the result.

Create a Calculator object in the main() method, call the add() method, and display the returned result.

Task: Demonstrate how a method can receive parameters, perform an operation, and return a value.

4. Rectangle Area Using a Method

Create a Java program to calculate the area of a rectangle. Define a class named Rectangle with two data members: length and width.

Create a method named area() that calculates and returns the area using:

Area = length $\times$ width

Create an object of the class, assign suitable values to length and width, call the area() method, and display the result.

Task: Demonstrate how a method can perform a calculation using the data members of an object and return the result.

5. Understanding Public and Private Access

A university wants to demonstrate how access control works in Java. Create a class named Student containing two data members: name and id.

- Declare name as public.
- Declare id as private.

Create another class containing the main() method. Create a Student object and try to access both variables from the other class.

Observe and demonstrate that the public member can be accessed directly, while the private member cannot be accessed directly outside its class.

Task: Write a Java program to demonstrate the difference between public and private access specifiers.

6. Demonstrating the Protected Access Specifier

Consider a university system where a Person class contains common information and a Student class inherits from it.

Create a superclass named Person containing a protected variable named name and a method displayName() to display it. Create a subclass named Student that extends Person.

Inside the Student class, access the protected variable and display the student's name.

Task: Write a Java program to demonstrate how the protected access specifier allows a subclass to access a member of its superclass.

7. Encapsulation Using a Student Class

A university wants to protect student information from being directly modified by other classes. Create a class named Student with a private data member name.

To access the private variable, create:

- setName() — to assign a student's name.
- getName() — to retrieve the student's name.

In the main() method, create a Student object, use the setter method to assign a name, and use the getter method to display the name.

Task: Write a Java program to demonstrate encapsulation using private data and getter/setter methods.

8. Encapsulation in a Bank Account

A bank wants to prevent its customers' account balance from being directly modified by other classes. Create a class named BankAccount with a private data member balance.

Create the following methods:

- deposit() — adds an amount to the balance.
- getBalance() — returns the current balance.

In the main() method, create a BankAccount object, deposit some amount, and display the current balance using the getter method.

The balance should not be accessed or modified directly from the main() method.

Task: Demonstrate how encapsulation protects data by restricting direct access to a private variable.

9. Animal and Dog Using Inheritance

A programmer wants to represent common animal behavior using inheritance. Create a superclass named Animal containing a method eat() that displays "Animal is eating".

Create a subclass named Dog that extends Animal and contains another method named bark() that displays "Dog is barking".

In the main() method, create a Dog object and call both eat() and bark().

Task: Demonstrate how a subclass can inherit and use a method from its superclass while also having its own method.

10. Vehicle and Car Using Inheritance

A vehicle management program needs to represent common vehicle behavior. Create a superclass named Vehicle with a method start() that displays a suitable message.

Create a subclass named Car that inherits from Vehicle and contains an additional method named drive().

Create a Car object in the main() method and call both the inherited start() method and the drive() method.

Task: Demonstrate how inheritance allows a subclass to reuse functionality from its superclass.

11. Person and Student Using Inheritance

A university system stores common information about people and additional information about students. Create a superclass named Person containing a data member name and a method displayName().

Create a subclass named Student that inherits from Person and contains an additional data member id.

Create a Student object, assign values to the student's name and ID, and display both pieces of information.

Task: Demonstrate how a subclass can inherit data and methods from a superclass while adding its own data members.

12. Method Overloading Using Calculator

A calculator application needs to perform addition with different numbers of values. Create a class named Calculator containing two methods with the same name add().

The first add() method should accept two integers and return their sum.

The second add() method should accept three integers and return their sum.

In the main() method, create a Calculator object and call both versions of the add() method with appropriate arguments.

Task: Demonstrate method overloading, where multiple methods have the same name but different parameter lists.

13. Method Overloading Using Display

A Java program needs to display different types of data using the same method name. Create a class named Display with three overloaded methods named show():

- One method should accept an int.
- One method should accept a double.
- One method should accept a String.

Each method should display the received value.

Create a Display object and call all three versions of the show() method.

Task: Demonstrate how method overloading allows the same method name to work with different types of parameters.

14. Method Overriding Using Animal and Dog

A software developer wants different animals to produce different sounds. Create a superclass named Animal containing a method sound() that displays a general animal sound message.

Create a subclass named Dog that overrides the sound() method and displays "Dog barks" instead.

Create a Dog object and call the sound() method.

Task: Demonstrate method overriding, where a subclass provides its own implementation of a method already defined in the superclass.

15. Method Overriding Using Shapes

A graphics application needs to represent different shapes. Create a superclass named Shape containing a method draw() that displays a general message such as "Drawing a shape".

Create two subclasses:

- Circle
- Rectangle

Both subclasses should override the draw() method and display their own messages, such as "Drawing a circle" and "Drawing a rectangle".

Create objects of both subclasses and call their draw() methods.

Task: Demonstrate method overriding by allowing different subclasses to provide their own implementation of the same method.